

Diseño conceptual y lógico de la base de datos del proyecto

Presentado por:

Edgar Andrés Romero Otálora
Julian David Miranda Leguizamón
Sebastian Bedoya Florez
Jhon Alexander Tenjo Romero

Contexto

Ecommify es una plataforma de comercio electrónico que conecta vendedores y compradores en un entorno digital centralizado. Permite gestionar productos, pedidos, pagos, entregas y experiencias de compra de manera eficiente. Además, incorpora análisis geográficos y de comportamiento para identificar patrones de ventas y clientes. Para soportar su crecimiento y alto volumen de operaciones, implementará una arquitectura escalable basada en una estrategia **Políglota Híbrida**.

Requisitos

Funcionales:

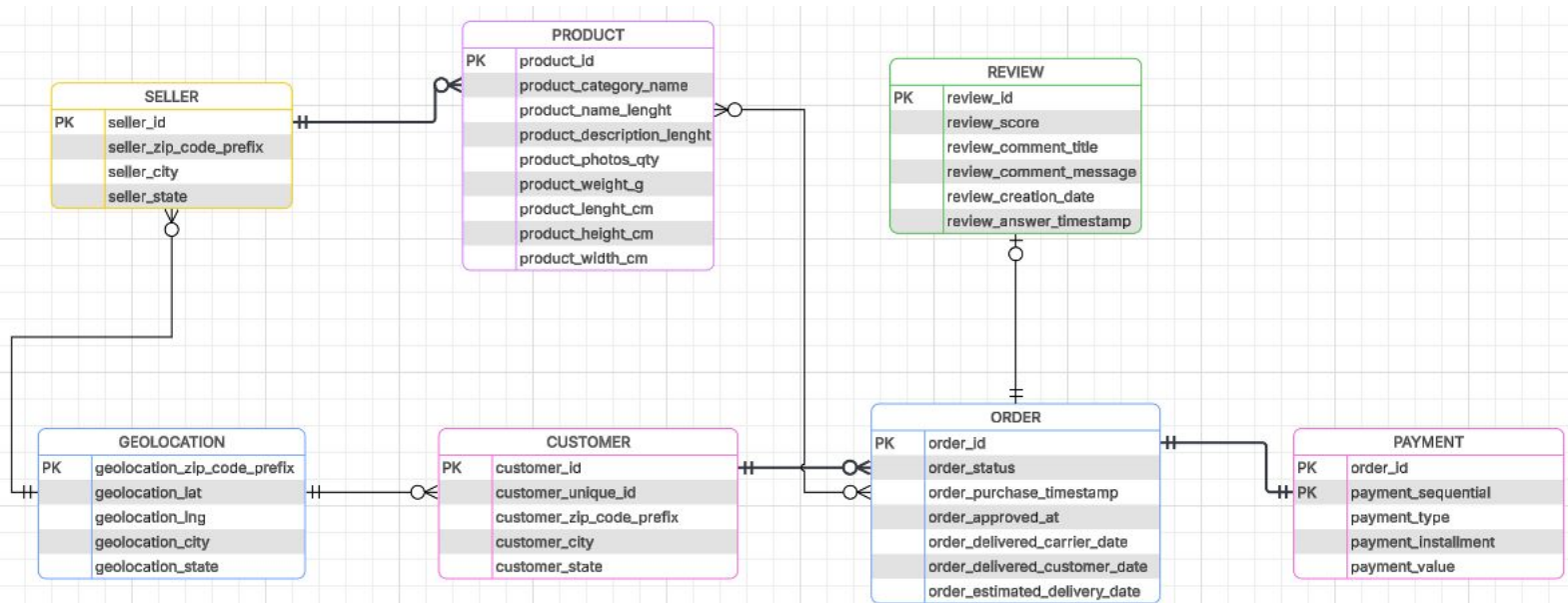
- Gestionar clientes y su información geográfica.
- Registrar y administrar órdenes de compra.
- Gestionar productos y categorías.
- Administrar vendedores y marketplaces asociados.
- Registrar pagos y métodos de pago.
- Gestionar ítems asociados a cada orden.
- Registrar reseñas y calificaciones de clientes.
- Permitir consultas analíticas sobre ventas, entregas y satisfacción.
- Generar reportes históricos y dashboards.
- Relacionar información geográfica mediante códigos postales y coordenadas.

Requisitos

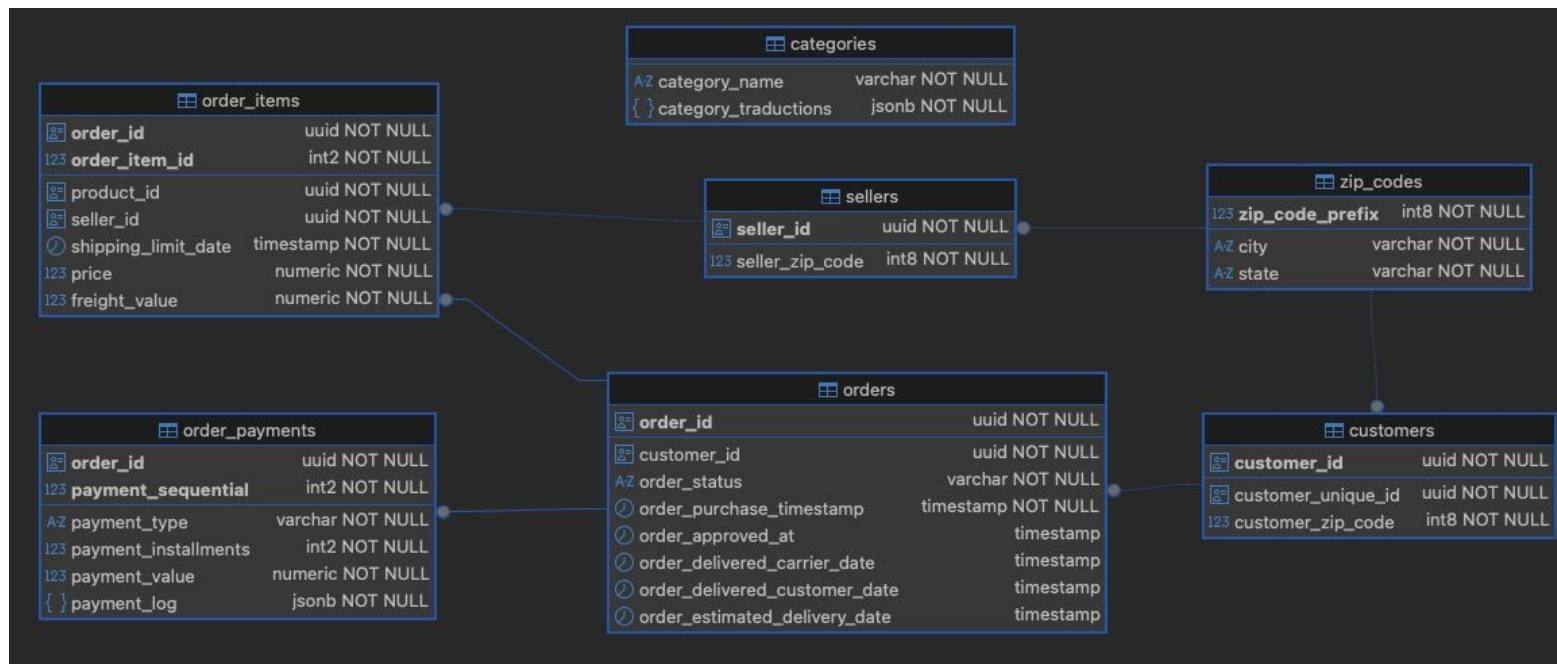
No Funcionales:

- Soportar consultas analíticas sobre más de 1 millón de registros geográficos
- Permitir crecimiento horizontal de datos y servicios
- Mantener consistencia entre órdenes, pagos y productos
- Arquitectura modular basada en microservicios y BD híbrida
- Optimizar operaciones OLAP y generación de reportes
- Integración con PostgreSQL y motores NoSQL
- Garantizar acceso continuo a órdenes y pagos

Diagrama Entidad - Relación



Esquema Relacional



Tipos de datos

UUID: es un identificador único universal usado para distinguir registros.

Varchar: almacena cadenas de texto de longitud variable dentro de bases.

INT2: almacena números enteros pequeños usando menos espacio de almacenamiento.

INT8: almacena números enteros grandes con alta capacidad de almacenamiento.

NUMERIC: almacena números decimales exactos con precisión definida y configurable.

JSONB: almacena datos JSON binarios optimizados para consultas y rendimiento.

TIMESTAMP: almacena fechas y horas exactas para registros temporales digitales.

Extensiones

UUID-OSSP: Es una extensión para generar identificadores UUID automáticamente en PostgreSQL.

PG_TRGM: es una extensión para búsquedas rápidas usando similitud textual avanzada.

Esquema MongoDB

Products

```
{
  "_id": "060cb19345d90064d1015407193c6ffc",
  "product_category_name": "perfumeria",
  "physical_specs": {
    "weight_g": 525,
    "length_cm": 17,
    "height_cm": 11,
    "width_cm": 12
  },
  "multimedia_metadata": {
    "product_name_length": 42,
    "product_description_length": 774,
    "photos_qty": 2
  },
  "performance_metrics": {
    "average_review_score": 4.6,
    "total_reviews_count": 18
  }
}
```

Esquema MongoDB

Order reviews

```
{
  "_id": "7ac549e1e29c625d3934329e43b546e5",
  "order_id": "e481f51cbdc54678b7349021f2804312",
  "review_score": 5,
  "review_comment": {
    "title": "Excelente servicio",
    "message": "El producto llegó mucho antes de lo estimado y en perfectas condiciones."
  },
  "timestamps": {
    "creation_date": "2017-09-21T00:00:00Z",
    "answer_timestamp": "2017-09-22T10:57:03Z"
  }
}
```

Esquema MongoDB

Geolocation

```
{
  "_id": "01001",
  "geolocation_zip_code_prefix": 1001,
  "location": {
    "type": "Point",
    "coordinates": [-46.634007, -23.549807]
  },
  "city": "sao paulo",
  "state": "SP"
}
```

Matriz de Decisiones de Almacenamiento SQL (PostgreSQL)

Colección	Estructura	Volumen y Velocidad	Teorema CAP
customers	Estructura altamente relacional. Contiene identificadores únicos de clientes, ciudad, estado y código postal. Relacionada directamente con orders.	Volumen medio (~99k registros). Velocidad media de lectura y escritura debido a consultas frecuentes de clientes y ubicación.	(C) PostgreSQL prioriza consistencia para garantizar integridad de datos de clientes y relaciones con pedidos.
orders	Tabla central transaccional con múltiples relaciones hacia customers, payments y order_items. Maneja estados y fechas del ciclo de vida del pedido.	Alto volumen (~99k pedidos) y alta velocidad transaccional. Escrituras frecuentes y consultas constantes.	(C) La consistencia es crítica para evitar pérdida o duplicidad de pedidos durante transacciones.
order_items	Estructura relacional detallada por producto vendido. Relaciona órdenes, productos y vendedores.	Alto volumen (~112k registros). Alta velocidad de inserción y consultas analíticas sobre ventas.	(C) Debe garantizar integridad entre pedidos, productos y vendedores.
order_payments	Datos financieros estructurados asociados a órdenes. Incluye cuotas, tipo de pago y montos.	Volumen alto (~103k registros). Escrituras moderadas y lecturas frecuentes para análisis financiero.	(C) Se prioriza consistencia para asegurar exactitud de pagos y facturación.
sellers	Información estructurada de vendedores con ubicación geográfica. Relacionada con order_items.	Volumen bajo (~3k vendedores). Lecturas frecuentes y pocas actualizaciones.	(C) Se requiere consistencia para mantener referencias correctas en ventas y logística.
categories	Tabla pequeña de traducción de categorías entre portugués e inglés. Relación auxiliar con products.	Volumen muy bajo (71 registros). Baja frecuencia de acceso.	(CA) Puede mantenerse altamente disponible y consistente debido a su tamaño reducido y baja complejidad.

Matriz de Decisiones de Almacenamiento NoSQL (MongoDB)

Colección	Estructura	Volumen y Velocidad	Teorema CAP
products	Estructura polimórfica: Los productos varían sus atributos según su categoría (ej: ropa utiliza tallas).	Volumen: Medio (~32k registros). Velocidad: Extremadamente Alta (Lecturas masivas concurrentes).	(P) El sistema está diseñado para sobrevivir a una división del clúster. (A) Ante dicha partición, el catálogo debe seguir respondiendo y mostrando los productos.
order_reviews	Estructura semi-estructurada: Documentos independientes vinculados por product_id. No todas las reseñas tienen la misma estructura.	Volumen: Alto (+100k registros y crece con cada orden). Velocidad: Media en Escritura y Alta en Lectura.	(P) Si la red se fragmenta entre centros de datos, los servidores de base de datos aislados siguen operando de forma autónoma. (A) Un cliente debe poder escribir o leer comentarios siempre. La caída de un enlace de red no puede bloquear la sección de opiniones.
geolocation	Estructura documental especializada (GeoJSON): Almacena puntos espaciales definidos por arreglos de coordenadas [longitud, latitud].	Volumen: Alto (Más de 1 millón de filas en el dataset original). Velocidad: Alta (Cálculos de proximidad en milisegundos para estimación de costo de envío durante el checkout).	(P) MongoDB maneja esta carga mediante Sharding. Si un nodo fragmentado regional sufre una desconexión, los demás shards siguen atendiendo consultas de sus respectivas regiones. (A) El cálculo del costo de envío en el carrito debe responder en milisegundos para evitar que el usuario abandone la compra.

Análisis del teorema CAP

Sacrificio de la Consistencia Estricta (C)

En las colecciones de products y order_reviews forzar una consistencia lineal exigiría que, ante una actualización de precio o creación de un comentario, todas las réplicas se bloqueen hasta confirmar el cambio. Esto degradará la velocidad de la plataforma drásticamente.

Se decide optar por un modelo de consistencia eventual sincronizando las réplicas de forma asíncrona a través del Message Broker.

Resiliencia ante Particiones de Red (T)

En un entorno distribuido en la nube los fallos son inevitables, cuando alguna comunicación entre particiones se interrumpe, MongoDB prioriza mantenerse operativo y responder a lecturas desde nodos secundarios guardando y protegiendo así la disponibilidad.

Arquitectura Híbrida

Para implementar una arquitectura híbrida políglota eficiente, se define un modelo de Sincronización Circular Asíncrona Orientada a Eventos (Event-Driven) entre PostgreSQL y MongoDB. Este enfoque asegura que ambos motores cooperen e intercambien información sin bloquearse mutuamente, manteniendo consistencia, escalabilidad y alto rendimiento en la plataforma.

Trade-offs y estrategias de mitigación

Al adoptar un enfoque arquitectónico AP en MongoDB para optimizar las consultas masivas de la plataforma, se asume y mitiga los siguientes trade-offs:

- *Pérdida de Consistencia Inmediata*
- *Duplicidad de datos y mayor consumo de almacenamiento*

Trade-offs y estrategias de mitigación

Pérdida de Consistencia Inmediata

Riesgo

Al priorizar la disponibilidad masiva (A), si un administrador actualiza el nombre o la categoría de un producto en el Write Model (PostgreSQL), un cliente que navegue por el catálogo NoSQL en ese mismo instante podría visualizar los datos anteriores durante una ventana de milisegundos mientras se propaga el cambio.

Estrategia de Mitigación

Este riesgo se mitiga mediante la implementación de Consistencia Eventual con Apache Kafka. Dado que el catálogo, la geolocalización y las opiniones no impactan directamente los balances contables de la empresa ni los flujos de facturación, el negocio tolera esta brecha temporal corta.

Trade-offs y estrategias de mitigación

Duplicidad de datos y mayor consumo de almacenamiento

Riesgo:

Al aplicar desnormalización por embedding en las colecciones de MongoDB para eliminar los costosos JOINS, la información estructural repetitiva eleva de forma drástica el consumo de almacenamiento en disco en comparación con un esquema relacional puro.

Estrategia de Mitigación

Se implementa una política estricta de Referenciación Ligera. Como se definió en el diseño de los JSONs, se embeben únicamente las dimensiones estáticas y las métricas precalculadas. Atributos pesados o transversales, como los diccionarios globales de traducción idiomática, no se duplican en MongoDB; en su lugar, se mantiene el string `product_category_name` como puente relacional, dejando que la capa de servicios o el API Gateway ensamble la traducción dinámicamente en memoria durante la capa de presentación.

Flujos de Sincronización

Sincronización Transaccional a Consulta (Ida: PostgreSQL → MongoDB)

Cada vez que se ejecuta un cambio de estado importante en el Write Model (eje. una actualización administrativa de categorías), PostgreSQL confirma el cambio de manera ACID. Inmediatamente después del commit, el servicio correspondiente publica un evento en el Message Broker (Kafka). El microservicio del catálogo consume este mensaje en el trasfondo y realiza un update sobre el campo correspondiente en el documento NoSQL de MongoDB, actualizando el catálogo de forma transparente.

Flujos de Sincronización

Sincronización de Interacción a Analítica (Vuelta: MongoDB → PostgreSQL):

Cuando un comprador genera datos que no alteran el flujo contable pero sí afectan al negocio como la inserción de una calificación de 5 estrellas en `order_reviews`, MongoDB persiste el documento de inmediato para asegurar baja latencia.

En paralelo, un proceso de captura emite un evento del tipo `ReviewSubmitted` hacia Kafka. El motor consumidor de PostgreSQL toma este payload de forma asíncrona y actualiza la reputación acumulada del vendedor en la tabla relacional, protegiendo a la base de datos transaccional de la concurrencia masiva del front-end.